

How BloodLink works, and how to test all of it

A blood-donation platform built around registered **blood banks**. Four roles, one spine: a recipient asks for blood, an officer matches the nearest eligible donor and records the donation into their bank's stock, and donors and recipients locate and get driving directions to the nearest bank. This manual walks each role end to end, maps how the pieces connect, and gives you a role-by-role QA script.

01 / The cast

Four roles, at a glance

Everyone signs in through one of two doors — the admin portal (`login.php`) or the shared user portal (`user-login.php` , role dropdown). The session `role` gates every page.

SESSION ROLE: ADMIN

Administrator

Runs the bank network: manages blood banks, donors, officers, recipients, and per-bank stock; posts emergency alerts; monitors the request queue; reads reports and the map.

SESSION ROLE: SUPERVISOR

Hospital Officer

Belongs to a blood bank. Works the request queue, matches each request to the nearest eligible donor, and **confirms the donation** — which banks the units into their own bank's stock.

SESSION ROLE: STUDENT

Blood Donor

Registers with a blood group and address, checks eligibility (56-day rule), sees emergency alerts, and uses the locator to find and route to the nearest bank.

SESSION ROLE: RECIPIENT

Recipient

Registers, files blood requests (urgency-flagged), tracks each request's status and any donation received, and uses the locator.

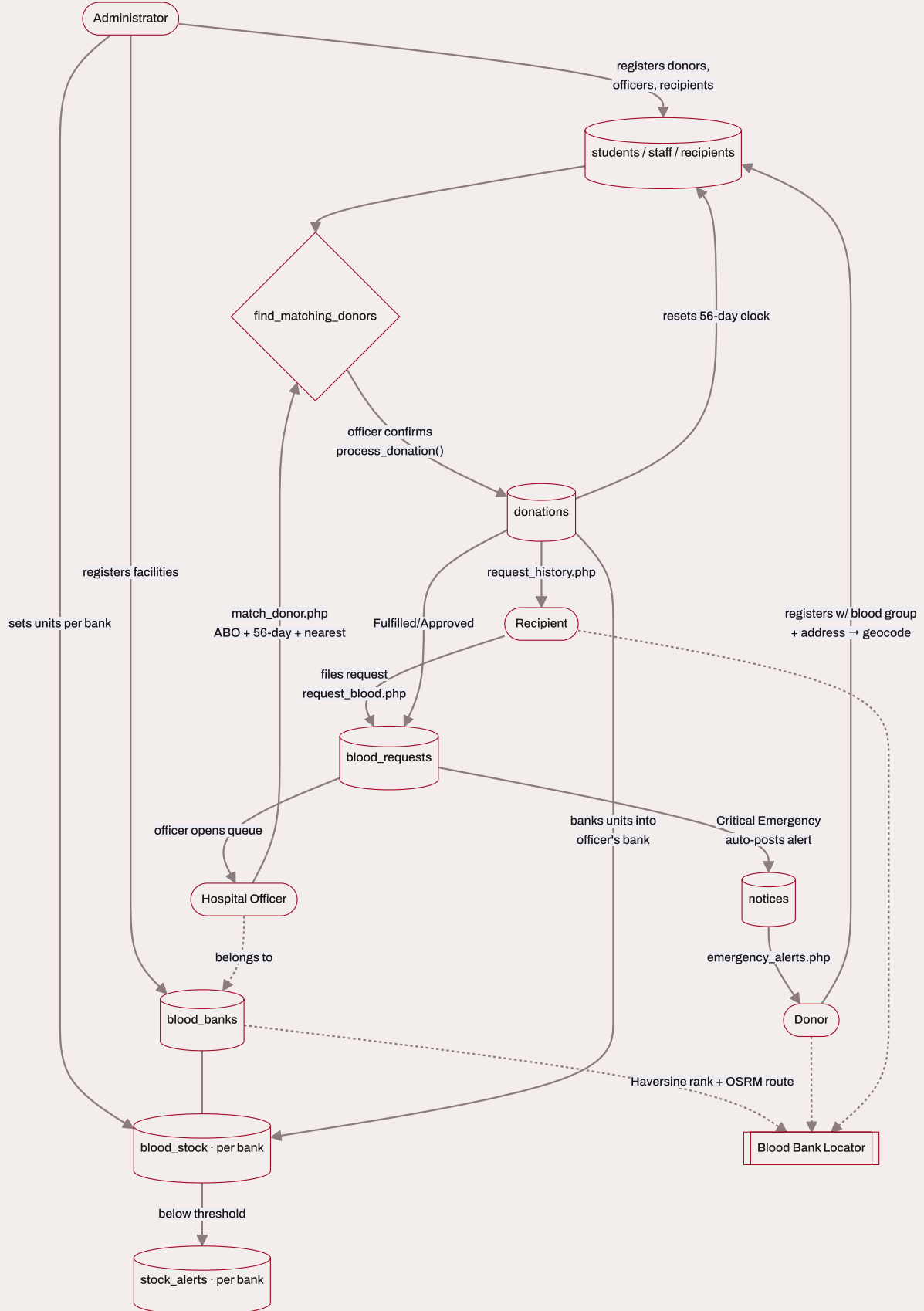
WHY "STUDENT / SUPERVISOR" IN THE CODE?

BloodLink began as a student-supervision (SIWES) project. Two table names kept the old words — `students` = donors, `staff` = officers — but every student-era feature (logbook, attendance, seminars, chat, donor-to-officer assignment) has been removed. It is now purely a blood-bank app; the leftover names are cosmetic.

02 / The spine

How the whole app connects

Everything orbits one workflow: a request becomes a matched, recorded donation banked at a specific facility. The locator, the 56-day rule and per-bank stock all hang off the blood-bank entity.



- **Admins** register the `blood_banks` (facilities), the people, and each bank's `blood_stock`.
- **Recipients** create `blood_requests`; a Critical Emergency also writes a donor-facing `notices` alert.
- **Officers** belong to a bank. `match_donor.php` runs `find_matching_donors()` (ABO/Rh × 56-day × nearest); `process_donation()` writes the `donations` row in one transaction that resets the donor's clock, **banks the units into the officer's bank**, updates the request, and clears that bank's alert.
- **Donors & recipients** use the **Blood Bank Locator**: it ranks banks nearest-first and draws an OSRM driving route to the chosen one.

03 / Data model

Ten tables, and who touches them

TABLE	HOLDS	WRITTEN BY	READ BY
login	Admin accounts	seed	admin login
blood_banks	Facilities (name, address, geocoded lat/lng, contact)	admin (manage banks)	locator, stock, officers, reports
staff	Officers (+ blood_bank_id)	admin (add officer → picks a bank)	officer login, matching, reports
students	Donors (+ blood_group, last_donation_date, lat/lng)	donor reg, admin add donor	matching, map, directory, reports
recipients	Patients (+ bcrypt password, lat/lng)	recipient reg, admin add recipient	recipient login, requests
blood_requests	Requests (+ urgency, status, lat/lng)	recipient (request_blood)	officer queue, match, history
donations	Donation ledger (+ blood_bank_id)	officer (process_donation)	histories, reports
blood_stock	Units per bank per blood type	admin, process_donation	stock page, dashboard, locator, reports
stock_alerts	Low-stock alerts per bank per type	stock page, process_donation	dashboard, stock page
notices	Emergency alerts	officer/admin, auto (critical request)	donor emergency alerts

This matches the brief's ERD (Figure 5.3): the design centres on `blood_banks`, with `blood_stock` and `stock_alerts` scoped per bank and `staff` belonging to a bank.

04 / Role walkthroughs

What each role does, end to end

ADMINISTRATOR runs the network

Signs in at `login.php` (`admin` / `admin`) → `dashboard.php` .

- **Blood Banks:** add / edit / delete facilities — each address is geocoded onto the map, and a new bank gets an empty stock row per blood type automatically.
- **People:** Manage Donors, Hospital Officers (each assigned to a bank), Recipients — full CRUD; officer & recipient passwords hashed on create.
- **Blood Stock:** pick a bank, then set units + low-stock threshold per type; dropping below threshold raises that bank's alert, restocking resolves it.
- **Oversight:** the Blood Requests queue (read-only for admin), Emergency Alerts, the Geo-Map/locator, and Reports (donations, blood-group spread, top donors/officers, CSV export).

HOSPITAL OFFICER makes the match

Signs in at `user-login.php` (Hospital Officer, email + password) → `officer_dashboard.php` . Each officer belongs to one bank.

- **Donors:** the donor directory — blood group and current 56-day eligibility.
- **Blood Requests:** the open queue, most urgent first. "Match Donor" opens `match_donor.php` .
- **Match & confirm:** only ABO/Rh-compatible, 56-day-eligible donors appear, **nearest first** (km from the request's hospital). Confirming runs `process_donation()` , which banks the units into *this officer's bank*.
- **Emergency Alerts & Geo-Map.**

BLOOD DONOR is the supply

Registers at `register.php` (blood group + geocoded address), signs in with Donor ID + password → `donor_dashboard.php` .

- **Donate Blood:** shows eligibility (inside/outside the 56-day window). Officers record the actual donation on match.
- **Emergency Alerts and the Blood Bank Locator** (Nearby Banks).

Registers, signs in with email + password → `recipient_dashboard.php`.

- **Request Blood:** patient, group, units, hospital, urgency. Critical Emergency auto-posts an alert; one pending request per group is enforced; the hospital is geocoded.
- **Request History** (status + donations received) and the **Locator**.

05 / Blood Bank Locator

The interactive map (brief Fig 5.5)

The locator answers "which registered bank is nearest, and how do I get there?" — proximity only; stock is managed separately.

1

Capture location

"Use my location" (browser GPS) or "Use Kaduna city centre". A blue marker shows where you are.

2

Rank the banks

Each bank's distance is computed with the Haversine formula and the list is sorted nearest-first, with a colored distance badge (green ≤5 km, amber 5–15, red >15) and dashed connectors to the three nearest.

3

Route to one

Click a bank's "Directions": OSRM returns a driving route drawn as a polyline **colored by distance band**, and the route box shows **distance by road + estimated drive time**.

Built entirely on free, open services — Leaflet + OpenStreetMap tiles, Nominatim geocoding, OSRM routing — so there is no API key or billing account, unlike Google Maps.

06 / The end-to-end donation

Follow one unit of blood

- 1 **Recipient files a Critical O- request**
→ a `blood_requests` row (Pending) + an auto `notices` alert.
- 2 **Donors see the alert**
on their Emergency Alerts page.
- 3 **Officer opens the queue**
`blood_requests.php` lists it top (Critical first) → Match Donor.
- 4 **System proposes donors**
only O-, 56-day-eligible, ordered by distance to the hospital.
- 5 **Officer confirms**
`process_donation()`: writes `donations` (with the officer's `blood_bank_id`), resets the donor's clock, adds units to that bank's `blood_stock`, flips the request, resolves that bank's alert.
- 6 **Everyone sees the result**
Recipient history shows Fulfilled; the donor is inside a fresh 56-day window; the bank's stock and the admin dashboard update.

07 / QA runbook

How to test the whole app

Set up once, then run each role's script. The seed is built for this — 5 real banks, donors on the 56-day boundary, all eight blood groups.

SETUP

```
mysql -u root -proot -e "CREATE DATABASE IF NOT EXISTS donor_app"
mysql -u root -proot donor_app < db/schema.sql
mysql -u root -proot donor_app < db/seed.sql
# app under Local: http://<site>.local/bloodlink/ · re-run seed.sql to reset
```

ROLE	PAGE	USERNAME	PASSWORD
Admin	login.php	admin	admin
Officer	user-login.php	maryam@bloodlink.test	Officer@123
Donor	user-login.php	BL-KD-001	Donor@123
Recipient	user-login.php	hassan@bloodlink.test	Recipient@123

A. AUTHENTICATION & ACCESS

- ✓ All 4 logins land on the right dashboard; wrong passwords rejected.
- ✓ Logged out, a protected page redirects (302); a donor can't reach an admin/officer URL.
- ✓ Logout works from the topbar avatar dropdown and the sidebar, every role.

B. ADMINISTRATOR

- ✓ **Blood Banks:** add a bank (address geocoded), edit it, delete it; a new bank shows 8 stock rows.
- ✓ Add a donor (real blood type shows, not "N/A"), an officer (assigned to a bank), a recipient — each new officer/recipient can then log in.
- ✓ **Blood Stock:** choose a bank; drop A+ below threshold → that bank's alert appears + dashboard count rises; save again → no duplicate; restock → resolves.
- ✓ Reports render; Total Donations equals the donations count; CSV export downloads.

C. RECIPIENT → OFFICER → DONOR (THE SPINE)

1

Recipient files O-, Critical Emergency

- ✓ Shows Pending in Request History; a matching alert exists; duplicate/invalid submissions refused.

2

Officer matches

- ✓ Only O- donors; nearest-first with a km distance; day-30 donor excluded, day-56 included.

3

Officer confirms

- ✓ Request → Fulfilled; the officer's bank's O- stock changes; the donation row carries that bank_id; re-confirming the same donor is refused.

D. BLOOD BANK LOCATOR

- ✓ "Use Kaduna city centre" ranks banks nearest-first with colored distance badges.

- ✓ Click "Directions" → a colored road route draws and the box shows distance + drive time.

- ✓ OpenStreetMap attribution is visible; banks show units in stock in their popup.

E. CROSS-CUTTING UI/UX

- ✓ No content sits under the fixed topbar/sidebar; tables scroll rather than stretch.

- ✓ Consistent chrome across all logged-in pages, all four roles.

- ✓ Accented text renders correctly (e.g. "Officer – Hospital"); phone width collapses the sidebar to a hamburger.

- ✓ A tampered/expired form token is rejected (CSRF).

KNOWN LIMITATIONS (EXPECTED)

- **Nominatim rate limit (1/sec):** registering banks/people rapidly may store NULL coordinates — they won't appear on the map until re-saved.
- **OSRM demo server** is fine for a demo, not production traffic; self-host it for real deployment.
- **Admins can't confirm donations** by design — the officer is the clinical actor, and the donation's bank comes from the officer's bank.
- **Rotate the old Google Maps key** in the Google console — it's out of the code but in earlier git history.

BloodLink Operations & QA Manual (v3) · roles: admin / supervisor / student / recipient · spine: blood_requests → find_matching_donors → process_donation → donations (banked per facility) · locator on OpenStreetMap + Nominatim + OSRM · 10 tables centred on blood_banks.